

Dockerfile and Container Lifecycle Handson

1. Check if docker is installed or not, on lab machine its already installed but if you are installing docker on another VM or machine, need to install it separately from

<https://docs.docker.com/engine/install/> .

```
(base) labuser@ip-172-31-27-175:~$ docker --version
Docker version 27.1.1, build 6312585
```

2. Create a sample [Dockerfile](#) using cat/nano/vi editor and save the contents.

```
(base) labuser@ip-172-31-27-175:~$ cat > Dockerfile
# 1. Base image
FROM nginx:alpine
# 2. Metadata about the image
LABEL description="Nginx web server serving static HTML and Docker"
# 4. Build-time temp variable (can be overridden at build time), this is not persisting inside container
ARG PAGE_TITLE="Welcome to Dockerized Nginx"
# 5. Set working directory, container will start with this Home directory
WORKDIR /usr/share/nginx/html
# 6. Copy custom HTML file to web root
COPY index.html /usr/share/nginx/html/
# 3. Set environment variable. Can be override during docker run.
ENV KUBECTL_VERSION=v1.29.0
# 7. ADD can copy additional Remote URL and auto extract .tar file. Like kubectl
ADD https://github.com/jqlang/jq/releases/download/jq-1.8.1/jq-1.8.1.tar.gz /tmp/
# 8. Declare a volume for persistent/static content bound to host machine (optional)
VOLUME ["/usr/share/nginx/html"]
# 9. Run a command during build
RUN mkdir -p /var/cache/nginx/client_temp /run && \
    chown -R nginx:nginx /var/cache/nginx /run
#10. Set a custom user (nginx runs as this user by default, can be checked with "user" command in container.)
USER root
# 11. Document exposed port (used by Nginx)
EXPOSE 80
# 12. Use custom entrypoint (overrides default, optional)
ENTRYPOINT ["/bin/sh", "-c"]
# 13. Default command (print a welcome message and start nginx)
CMD ["nginx -g 'daemon off;']
```

3. We also need index.html to host on the nginx server inside container. Create a sample index.html file with minimal content (can be modified accordingly).

```
(base) labuser@ip-172-31-27-175:~$ cat > index.html
Test html webpage for Dockerfile example
```

4. Now since we have the Dockerfile ready we can start building the docker image, which we will later use to launch container(s).
5. Use below command format to start building the docker image :

docker build -t <IMAGENAME>:<tag-name> .

docker build -t docker-example:latest . # If dockerfile is in current directory

```
(base) labuser@ip-172-31-27-175:~$ docker build -t docker-example:latest .
[+] Building 5.3s (11/11) FINISHED
=> [internal] load build definition from Dockerfile
=> => transferring dockerfile: 1.37kB
=> [internal] load metadata for docker.io/library/nginx:alpine
=> [internal] load .dockerignore
=> => transferring context: 2B
=> [1/5] FROM docker.io/library/nginx:alpine@sha256:d67ea0d64d518b1bb04acde3b00f722ac3e9764b3209a9b0a98924ba35e4b779
=> => resolve docker.io/library/nginx:alpine@sha256:d67ea0d64d518b1bb04acde3b00f722ac3e9764b3209a9b0a98924ba35e4b779
=> => sha256:fc17a5c08500c436f61d837fcb05441b359b3c9ec031058711bea5102f9e37cd 2.50kB / 2.50kB
=> => sha256:89c006ea7db85f1d449c0b687fa8a6ae5a53f95723fef7f6a29ce3fc0ebf03ce 10.80kB / 10.80kB
=> => sha256:6e174226ea690ced550e5641249a412cdbe2d09871f3e64ab52137a54ba606 4.13MB / 4.13MB
=> => sha256:f6bd92514b6f356d636a730d2c374892a63db0ba16205c9a4cd9bc7000863196 626B / 626B
=> => sha256:d67ea0d64d518b1bb04acde3b00f722ac3e9764b3209a9b0a98924ba35e4b779 10.33kB / 10.33kB
=> => sha256:ab753f997c88636c549e74dd1fc1f7990eb45392644b3fbc7cc6637131a5ce0a 1.80MB / 1.80MB
=> => sha256:1c81249bdabba0496b6c9537b6642d10232138a6f51d426b92ccfef6c35c4e21 954B / 954B
=> => extracting sha256:6e174226ea690ced550e5641249a412cdbe2d09871f3e64ab52137a54ba606
=> => sha256:ded4450e05847b33ef1fedd8d09aee29f395013b012e3b1e318b37b1ad82732f 404B / 404B
=> => sha256:9d51f8bf5f2b501c2c3b4df9ea08846aa5102eb92012c304e9db6cbc8e012da5 1.21kB / 1.21kB
=> => extracting sha256:ab753f997c88636c549e74dd1fc1f7990eb45392644b3fbc7cc6637131a5ce0a
=> => sha256:37c699686bc84a1a024b03ec8372a78c8df83161e39f2de56bbd41159641b0cf 1.40kB / 1.40kB
=> => extracting sha256:f6bd92514b6f356d636a730d2c374892a63db0ba16205c9a4cd9bc7000863196
=> => extracting sha256:1c81249bdabba0496b6c9537b6642d10232138a6f51d426b92ccfef6c35c4e21
=> => extracting sha256:ded4450e05847b33ef1fedd8d09aee29f395013b012e3b1e318b37b1ad82732f
=> => sha256:36f61153929d54ee8fb2cf94ecf693d27b1db4870982592ae7a4779e4259ecbe 16.92MB / 16.92MB
=> => extracting sha256:9d51f8bf5f2b501c2c3b4df9ea08846aa5102eb92012c304e9db6cbc8e012da5
=> => extracting sha256:37c699686bc84a1a024b03ec8372a78c8df83161e39f2de56bbd41159641b0cf
=> => extracting sha256:36f61153929d54ee8fb2cf94ecf693d27b1db4870982592ae7a4779e4259ecbe
=> [4/5] ADD https://github.com/jqlang/jq/releases/download/jq-1.8.1/jq-1.8.1.tar.gz /tmp/
=> [internal] load build context
=> => transferring context: 78B
=> [2/5] WORKDIR /usr/share/nginx/html
=> [3/5] COPY index.html /usr/share/nginx/html/
=> [4/5] ADD https://github.com/jqlang/jq/releases/download/jq-1.8.1/jq-1.8.1.tar.gz /tmp/
=> [5/5] RUN mkdir -p /var/cache/nginx/client_temp /run && chown -R nginx:nginx /var/cache/nginx /run
=> exporting to image
=> => exporting layers
=> => writing image sha256:147fffe35ca1a0124f9a843847d666d4d66523c7a76a81381073a3cb5189527f
=> => naming to docker.io/library/docker-example:latest
(base) labuser@ip-172-31-27-175:~$
```

6. Check if image is created, also note the IMAGE ID which is also displayed during image building.
Note : “latest” tag is created by default if we don’t provide a tag name in docker build command.

\$ docker images

```
=> exporting to image
=> => exporting layers
=> => writing image sha256:73285372f87c2a29df256ffddcc19fbd9f55dee72acab15b1a1f88dadf8d0690c
=> => naming to docker.io/library/docker-example:latest
(base) labuser@ip-172-31-27-175:~$ docker images
REPOSITORY          TAG             IMAGE ID        CREATED         SIZE
docker-example      latest         73285372f87c   18 seconds ago  55MB
(base) labuser@ip-172-31-27-175:~$
```

7. Run a container using this newly created image. `docker run -p 8081:80 docker-example`
We are instructing docker to use image “docker-example” and map our lab machine’s port 8081 to bind to container port 80. Our application should be accessible on port 8081 on our lab VM.

```
(base) labuser@ip-172-31-27-175:~$ docker run -p 8081:80 docker-example
2025/08/03 05:27:09 [notice] 1#1: using the "epoll" event method
2025/08/03 05:27:09 [notice] 1#1: nginx/1.29.0
2025/08/03 05:27:09 [notice] 1#1: built by gcc 14.2.0 (Alpine 14.2.0)
2025/08/03 05:27:09 [notice] 1#1: OS: Linux 6.5.0-1017-aws
2025/08/03 05:27:09 [notice] 1#1: getrlimit(RLIMIT_NOFILE): 1048576:1048576
2025/08/03 05:27:09 [notice] 1#1: start worker processes
2025/08/03 05:27:09 [notice] 1#1: start worker process 7
2025/08/03 05:27:09 [notice] 1#1: start worker process 8
2025/08/03 05:27:09 [notice] 1#1: start worker process 9
2025/08/03 05:27:09 [notice] 1#1: start worker process 10
```

Nginx application should start inside container and display some logs. Don’t quit the container yet.

8. Browse the URL **<LAB-MACHINE-IP>:8081** on a browser, it should display the html content defined in Step#3 above.



You can monitor the logs on the container, and you should see the activity when you browse the webpage.

```
2025/08/03 05:27:00 [notice] 1#1: start worker process 9
2025/08/03 05:27:09 [notice] 1#1: start worker process 10
119.252.206.169 - - [03/Aug/2025:05:27:42 +0000] "GET / HTTP/1.1" 304 0 "-" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/138.0.0.0 Safari/537.36" "-"
```

9. Press **CTRL+C** to stop & quit the container and watch for logs.

```
119.252.206.169 - - [03/Aug/2025:05:27:42 +0000] "GET / HTTP/1.1" 304 0 "-"
^C2025/08/03 05:28:04 [notice] 1#1: signal 2 (SIGINT) received, exiting
2025/08/03 05:28:04 [notice] 7#7: exiting
2025/08/03 05:28:04 [notice] 10#10: exiting
2025/08/03 05:28:04 [notice] 9#9: exiting
2025/08/03 05:28:04 [notice] 7#7: exit
2025/08/03 05:28:04 [notice] 10#10: exit
2025/08/03 05:28:04 [notice] 9#9: exit
2025/08/03 05:28:04 [notice] 8#8: exiting
2025/08/03 05:28:04 [notice] 8#8: exit
2025/08/03 05:28:04 [notice] 1#1: signal 17 (SIGCHLD) received from 9
2025/08/03 05:28:04 [notice] 1#1: worker process 9 exited with code 0
2025/08/03 05:28:04 [notice] 1#1: signal 29 (SIGIO) received
2025/08/03 05:28:04 [notice] 1#1: signal 17 (SIGCHLD) received from 10
2025/08/03 05:28:04 [notice] 1#1: worker process 10 exited with code 0
2025/08/03 05:28:04 [notice] 1#1: signal 29 (SIGIO) received
2025/08/03 05:28:04 [notice] 1#1: signal 17 (SIGCHLD) received from 7
2025/08/03 05:28:04 [notice] 1#1: worker process 7 exited with code 0
2025/08/03 05:28:04 [notice] 1#1: signal 29 (SIGIO) received
2025/08/03 05:28:04 [notice] 1#1: signal 17 (SIGCHLD) received from 8
2025/08/03 05:28:04 [notice] 1#1: worker process 8 exited with code 0
2025/08/03 05:28:04 [notice] 1#1: exit
(base) labuser@ip-172-31-27-175:~$
```

10. Check the status of all container (**RUNNING** + **STOPPED** + **CREATED** + **EXISTED** etc) using :
docker ps -a ## **docker ps** will only show **RUNNING** containers.

```
(base) labuser@ip-172-31-27-175:~$ docker ps -a
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS              PORTS          NAMES
adca0cb40a80   docker-example "/bin/sh -c 'nginx -...' About a minute ago Exited (0) 29 seconds ago                                peaceful_dewdney
```

11. We can launch the container in Detached/Background mode using :

docker run -d -p 8081:80 docker-example

```
(base) labuser@ip-172-31-27-175:~$ docker run -d -p 8081:80 docker-example
beb0d704b3daecc3f3b9648008a999932b10d47d8c3020059cd6a74f81f83d01
(base) labuser@ip-172-31-27-175:~$ docker ps -a
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS              PORTS          NAMES
beb0d704b3da   docker-example "/bin/sh -c 'nginx -...' 9 seconds ago   Up 8 seconds        0.0.0.0:8081->80/tcp, :::8081->80/tcp  bold_bhabha
adca0cb40a80   docker-example "/bin/sh -c 'nginx -...' 2 minutes ago   Exited (0) About a minute ago                                peaceful_dewdney
(base) labuser@ip-172-31-27-175:~$
```

12. Confirm the application status on the browser :



13. Since container is running in background, we can login into the container to check any files or logs etc :

docker exec -it <<CONTAINER-ID OR CONTAINER-NAME>> sh

```
(base) labuser@ip-172-31-27-175:~$ docker ps -a
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS                               NAMES
beb0d704b3da   docker-exampl "/bin/sh -c 'nginx -..." 9 seconds ago  Up 8 seconds  0.0.0.0:8081->80/tcp, :::8081->80/tcp  bold_bhabha
adca0cb40a80   docker-exampl "/bin/sh -c 'nginx -..." 2 minutes ago  Exited (0)    About a minute ago  peaceful_dewdney

(base) labuser@ip-172-31-27-175:~$ date
Sun Aug  3 05:32:57 UTC 2025
(base) labuser@ip-172-31-27-175:~$ docker exec -it beb0d704b3da sh
/usr/share/nginx/html #
/usr/share/nginx/html # ls -latr
total 20
-rw-r--r-- 1 root root      497 Jun 24 17:57 50x.html
drwxr-xr-x 1 root root     4096 Jul 15 19:55 .
-rw-rw-r-- 1 root root      41 Aug  3 04:17 index.html
drwxr-xr-x 2 root root     4096 Aug  3 05:29 ..
/usr/share/nginx/html # cat index.html
Test html webpage for Dockerfile example
/usr/share/nginx/html #
```

This will allow us to login inside the “sh” shell session of the container where we can see/edit/create files if required.

```
/usr/share/nginx/html # echo $0
sh
/usr/share/nginx/html # id
uid=0(root) gid=0(root) groups=0(root),1(bin),2(daemon),3(sys),4(adm),6(disk),10(wheel),11(floppy),20(dialout),26(tape),27(video)
/usr/share/nginx/html #
```

14. Now its time to stop/kill container via command (kill is like hard stop):

docker stop/kill <<CONTAINER-ID OR CONTAINER-NAME>>

docker ps -a # To confirm if above container is actually stopped.

```
(base) labuser@ip-172-31-27-175:~$ docker stop beb0d704b3da
beb0d704b3da
(base) labuser@ip-172-31-27-175:~$ docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS        NAMES
(base) labuser@ip-172-31-27-175:~$ docker ps -a
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS                               NAMES
beb0d704b3da   docker-exampl "/bin/sh -c 'nginx -..." 8 minutes ago  Exited (0)    42 seconds ago  bold_bhabha
adca0cb40a80   docker-exampl "/bin/sh -c 'nginx -..." 10 minutes ago Exited (0)    9 minutes ago  peaceful_dewdney
(base) labuser@ip-172-31-27-175:~$
```

If needed we can again START the previously CREATED/STOPPED container using :

docker start <<CONTAINER-ID OR CONTAINER-NAME>>

```
(base) labuser@ip-172-31-27-175:~$ docker start beb0d704b3da
beb0d704b3da
(base) labuser@ip-172-31-27-175:~$ docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS                               NAMES
beb0d704b3da   docker-exampl "/bin/sh -c 'nginx -..." 9 minutes ago  Up 3 seconds  0.0.0.0:8081->80/tcp, :::8081->80/tcp  bold_bhabha
(base) labuser@ip-172-31-27-175:~$
```

Note : We can also PAUSE/UNPAUSE the container if it is causing any trouble to our host machine, PAUSE will temporary release the resources used by container but will not STOP/KILL it. We can UNPAUSE the container without losing any data inside the container.

15. Time to do a final Stop using :

docker stop/kill <<CONTAINER-ID OR CONTAINER-NAME>>

```
(base) labuser@ip-172-31-27-175:~$ docker ps -a
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS                    NAMES
beb0d704b3da   docker-example "/bin/sh -c 'nginx -..." 19 minutes ago Up 10 minutes   0.0.0.0:8081->80/tcp, :::8081->80/tcp bold_bhabha
adca0cb40a80   docker-example "/bin/sh -c 'nginx -..." 22 minutes ago Exited (0) 21 minutes ago peaceful_dewdney
(base) labuser@ip-172-31-27-175:~$ docker stop beb0d704b3da
beb0d704b3da
(base) labuser@ip-172-31-27-175:~$
(base) labuser@ip-172-31-27-175:~$ docker ps -a
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS                    NAMES
beb0d704b3da   docker-example "/bin/sh -c 'nginx -..." 20 minutes ago Exited (0) 3 seconds ago bold_bhabha
adca0cb40a80   docker-example "/bin/sh -c 'nginx -..." 22 minutes ago Exited (0) 21 minutes ago peaceful_dewdney
```

16. Once STOPPED, we can REMOVE a container. RUNNING containers can't be removed.

docker rm <<CONTAINER1-ID/NAME>> <<<CONTAINER2-ID/NAME>> . . .

17. We will also delete the images created by us and PRUNE the garbage files/networks/volumes to clean up the storage on our virtual machines using PRUNE command:

docker images # Get the image ID

docker rmi <<IMAGEID>> OR <<IMAGE-NAME>>:<tag> # Delete image using imageID Or Name:tag

docker system/container/image/network/builder/volume prune

```
(base) labuser@ip-172-31-27-175:~$ docker rm beb0d704b3da adca0cb40a80
beb0d704b3da
adca0cb40a80
(base) labuser@ip-172-31-27-175:~$ docker ps -a
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS                    NAMES
(base) labuser@ip-172-31-27-175:~$ docker images
REPOSITORY      TAG            IMAGE ID          CREATED          SIZE
docker-example   latest        147fffe35ca1     24 minutes ago  55MB
(base) labuser@ip-172-31-27-175:~$
(base) labuser@ip-172-31-27-175:~$
(base) labuser@ip-172-31-27-175:~$ docker rmi docker-example
Untagged: docker-example:latest
Deleted: sha256:147fffe35ca1a0124f9a843847d666d4d66523c7a76a81381073a3cb5189527f
(base) labuser@ip-172-31-27-175:~$ docker images
REPOSITORY      TAG            IMAGE ID          CREATED          SIZE
(base) labuser@ip-172-31-27-175:~$
(base) labuser@ip-172-31-27-175:~$ docker system prune
WARNING! This will remove:
- all stopped containers
- all networks not used by at least one container
- all dangling images
- unused build cache

Are you sure you want to continue? [y/N] y
Deleted build cache objects:
ix7n1zkgesnvuy3rbuzryevrc
p9bh7i2p10k00izpjj3i9q36e
9hw4l0eh1954mwukc3s8qkyjs
ry60birdwxqj4azv9sw1p6vpj
wdar2prvk9a164yyzggujl5an
nqvqcmns6doj7d6rt4sla4vk8
mzh15rer6xwvbz3qtbvskp7l
t40x7texive3bprewtoumdti
k5n81g32l2sowemc1kcrndry
lxwp4on2vkh4z81bltcjg7c04
qeu9dys1uzt17m93j0wqj47g8
vie21pqc3om0qoldhg5celvn
iaq1bm8iidse6riaq6h9wptjz
fzsjz4vi1jhto11sittmt85et
tobq201s1hvez4huagv0uysydg
l08hg8p44m884asbss6hdkedv

Total reclaimed space: 4.055MB
(base) labuser@ip-172-31-27-175:~$
```